

# CPU ARCHITECTURE DESIGN

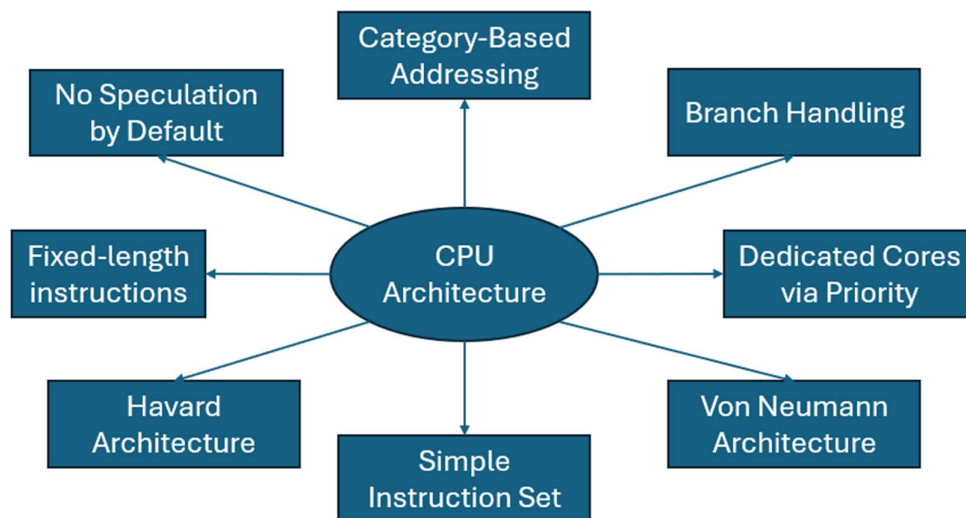
## Contents

|                                                                  |          |
|------------------------------------------------------------------|----------|
| <b>Introduction .....</b>                                        | <b>2</b> |
| <b>Category-based addressing &amp; Execution .....</b>           | <b>3</b> |
| <b>Instruction Set and CPU FDE Cycle.....</b>                    | <b>4</b> |
| <b>CPU Architecture Branch Handling and Priority System.....</b> | <b>5</b> |
| <b>CPU Workflow .....</b>                                        | <b>6</b> |
| <b>CPU Architecture Trade-Offs .....</b>                         | <b>7</b> |

## Introduction

This CPU Architecture is designed to improve the performance and security of CPUs. This architecture was also designed to address the current issues and limitations of the CPU. The CPU architecture was developed as currently CPUs aren't fit and designed to handle modern computer workloads. This was also developed to fit in and ensure full compatibility with my other designs. This CPU architecture will cover the FDE cycle, instruction set and branching handling. It will also show a workflow of what the CPU could look like in action along with trade-offs of this CPU architecture.

This CPU Architecture cannot promise perfect performance or security at any point during usage. However, this CPU architecture can improve efficiency, security and performance of CPUs across devices. This CPU architecture is also compatible with the Von Neumann architecture and Harvard architecture. This is just a design that engineers can use as reference and guide for implementing this new CPU architecture. Make sure to adapt features and systems based on resources and budget.



The spider diagram shows some of the key features of this CPU Architecture. We can clearly see that it is compatible with both current computing architectures. This spider diagram does not show all features involved in the CPU architecture. More details can be found in the sections below.

## **Category-based addressing & Execution**

- **Category-based addressing is where the data and instructions stored in cache and RAM are sorted into categories.**
- **Category-based addressing allows for the CPU to fetch instructions from memory more quickly leading to quicker execution.**
- **There are 4 components within the CPU that category-based addressing affects.**
- **The first component is the program counter otherwise known as PC.**
- **The program counter would need to be configured to support categories alongside addresses.**
- **For example, the program counter could display OS: 0x1000 as the next location of the instruction to be fetched from memory.**
- **This change is needed because it allows the CPU to know what data to fetch from cache or RAM.**
- **The second component is the memory address register also known as MAR.**
- **The memory address register would need to be configured to support temporary storage of both category and address.**
- **For example, the memory address register would hold the data OS: 0x1000 from the PC until the address bus uses it.**
- **This change is needed because it allows the CPU's address bus to collect the navigation data to find the instruction in memory.**
- **The third component is the address bus.**
- **The address bus would need to be configured to navigate categories in memory and then continue to find the instruction using the address.**
- **For example, the address bus would navigate to the OS category in RAM and then find the instruction using the address 0x1000.**
- **This change is needed because it allows the CPU to quickly access and find instructions in memory.**
- **The fourth component is the control unit.**
- **The control unit would need to be configured to manage the data flow between categories and execution priority.**
- **For example, the control unit may provide the OS category with the most CPU time and adapt to provide the application category with the most CPU time.**
- **This change is needed because it allows the CPU to allocate categories with CPU time depending on workload.**
- **The other components in the CPU such as the MDR, ALU, Accumulator and CIR do not require any changes for category-based addressing and execution.**
- **This is because the ALU manages calculations, the MDR stores the instruction, and the CIR also stores the instruction.**
- **The accumulator just stores the results of calculations performed by the ALU which isn't needed for category-based addressing and execution.**

## **Instruction Set and CPU FDE Cycle**

- An instruction set is how instructions are organized and formatted within memory before the CPU decodes them.
- There are a few changes that this CPU architecture requires to the instruction set.
- The first change is fixed-length instructions.
- This means that every instruction is the same size compared to x86 which uses variable-length instructions.
- This improves performance and fetch speed as the CPU knows exactly how many bytes to fetch from memory.
- The second change is the Opcode + Operand split within the instruction format.
- This means that the first x bits of the instruction are the opcode while the next y bits are the operand.
- For example, in a 32-bit instruction, the first 8 bits are opcode while the remaining 24 bits are operand.
- This will require changes to the compiler to ensure that code is successfully translated into fixed-length instructions.
- This improves performance and decode speed as the CPU doesn't need to figure how long an instruction is and only needs to split the instruction as it is already formatted simply.
- The FDE cycle still has the same 4 steps.
- The first step is fetching.
- This is where the CPU fetches instructions from RAM or cache by navigating through categories and address.
- The CPU uses the PC, MAR, address bus, data bus and MDR to perform the fetching of the FDE cycle.
- The second step is decoding.
- This is where the CPU decodes the instruction fetched from RAM or cache by splitting into operand and opcode.
- The CPU uses CIR to store the instruction while it is decoded.
- The third step is executing.
- This is where the CPU executes the instruction using the operand and opcode to perform functions.
- The CPU uses components such as the monitor, mouse, internal drive, GPU and ALU to execute instructions.
- The fourth and final step is repeating.
- This is where the CPU repeats the process of fetching, decoding and executing using the category-based addressing and execution.
- The CPU's program counter increments by 1 each time and points to the location of the next instruction in memory using the category and the address.
- This FDE cycle repeats forever until either the device is powered off or an issue

## **CPU Architecture Branch Handling and Priority System**

- The CPU currently uses branch handling to guess what the path of the next program is going to be.
- There are two options to branch handling that this CPU Architecture needs.
- The first option is no speculation by default.
- This means that the CPU must fetch data early before the CPU branching reaches the final breach.
- This then leads to the data arriving on time due to the speed of category-based addressing and allows the CPU to make the correct decision.
- However, this option may not be compatible with my CPU architecture as its fetching process can be quicker than branching.
- The second option is no branching handling.
- This means that the CPU doesn't use branching as it can fetch and decode instructions extremely quickly meaning less wait time.
- This then leads to no mispredictions meaning security vulnerabilities like Spectre and Meltdown are eliminated completely.
- This CPU architecture also uses a priority system for dedicating cores and CPU time.
- There are two types of priority systems that this CPU Architecture can use.
- The first type is dynamic priority.
- This is where the CPU dedicates core for each category and decides which category gets the most CPU time based on workload.
- This can be done through the CU as it can manage CPU time allocation in real-time while the operating system handles the core allocation.
- However, this dynamic priority can have latency issues as the cores must change categories depending on workload which may reduce performance.
- This dynamic priority improves adaptability and performance overall as it allows cores to focus on one category and allows CPU time to be managed in real-time.
- The second type is fixed priority.
- This is where the CPU dedicates cores for each category once and allocates CPU time fixedly with OS category processes getting the most CPU time.
- This can be done also through the CU as it can manage the CPU time allocation for each category, and the operating system can create fixed dedication for each core.
- However, this fixed priority can mean that the performance of applications or the operating system can slower as only one gains the most priority throughout CPU lifespan.
- This fixed priority improves performance as it allows cores to focus on one category and allocate CPU time where its commonly needed without latency issues like the dynamic priority can experience.

## **CPU Workflow**

- This CPU workflow will cover the process of CPU setup and FDE cycle during active use of a device.
- First, the CPU powers on for the first time and immediately fetches, decodes and executes instructions from RAM and internal drive to load the operating system.
- Second, the CU within the CPU gives the most CPU time to OS category processes on the device.
- This CPU uses the dynamic priority system which is currently assigning the most CPU time to operating system processes as no applications have been used.
- Third, the operating system dedicates the most CPU cores to OS category processes as currently the workload is the heaviest there.
- Fourth, the user installs GTA 6 and starts the application on their device.
- Fifth, the operating system allocates the most CPU cores to the application category processes as it is currently the heaviest workload in the system.
- Sixth, the CU gives the most CPU time to App category processes on the device.
- Seventh, the CPU repeatedly does the FDE cycle for instructions on the device.
- The CPU fetches the instruction using the PC, MAR, address bus, data bus and MDR.
- The program counter shows App: 0x3A00 which is the location of the next address within memory.
- The memory address register temporarily stores the location App: 0x3A00 until the address bus uses it.
- The address bus travels to cache checking the L1 cache, but it results in a cache miss, so the bus travels directly to RAM.
- The address bus then navigates to the app category and then locates the address 0x3A00 within the app category.
- The data bus then transports the instruction 0001 00001111 to the MDR for temporary storage
- The memory data register stores the instruction temporarily stores the instruction until it is transferred to the CIR.
- The CIR then holds the current instruction while the CPU decodes the instruction.
- The CPU decodes the instruction into opcode and operand to then be executed by other components.
- 0001 is decoded into ADD while 00001111 is decoded in R15 which the CPU uses the ALU to perform the calculation where needed.
- The CPU executes the instruction by using the ALU, storing the result in the accumulator or using the other components in the device.
- The results are then written directly back to application category within memory.
- The program counter increments by 1 and the next instruction App:0x4B20 within memory repeating the FDE cycle once again.

## **CPU Architecture Trade-Offs**

- **This CPU Architecture has trade-offs just like any other system I have created so far.**
- **There also may be additional implementation challenges that developers and engineers may face.**
- **I have identified 6 main trade-offs with this CPU Architecture.**
- **The first trade-off is heat.**
- **This trade-off exists because since the CPU can complete the FDE cycle more quickly and efficiently, it can also produce more heat.**
- **This means that the CPU may overheat if it works too quickly without limitations.**
- **This trade-off can be managed by cooling systems, clock speed limitations and heat monitoring to ensure the CPU doesn't overheat.**
- **The second trade-off is performance.**
- **This trade-off exists because dynamic priority can have latency delays and the Von Neumann architecture may be less effective due to the bottleneck of the bus.**
- **This trade-off can be managed by using Havard Architecture where possible, rate-limiting to ensure that cores do not change too frequently and accepting the limitations of the Von Neumann architecture.**
- **The third trade-off is cost.**
- **This trade-off exists because developing this CPU architecture requires coordination between OS vendors and hardware developers as well as materials and budget to build.**
- **This trade-off can be managed by using pre-existing agreements, budget management and testing the CPU architecture before full development.**
- **The fourth trade-off is time-consumption.**
- **This trade-off exists because developing this CPU architecture will take years due to the software and firmware changes required.**
- **This trade-off can be managed by structured development processes, role-based work and strong coordination between OS and hardware vendors.**
- **The fifth trade-off is security**
- **This trade-off exists because if this CPU architecture were to include branch handling, then security vulnerabilities like Spectre and Meltdown can be exploited.**
- **This trade-off can be managed by implementing the CPU architecture without the branching or accepting the risk of exploitation.**
- **The sixth trade-off is lack of legacy support**
- **This trade-off exists as this CPU architecture will mostly likely require a new generation of CPU.**
- **This trade-off can be managed by implementing the category-awareness through firmware/software updates for current devices and accepting that it won't be compatible for everyone.**